# ONEDRAFT

## CAD-AI — Aria Powered

### AI Intelligence & Model Provider Architecture

### Version 1.0

**Status:** Implementation Specification
**Subsystem:** AI Intelligence & Model Provider Architecture
**Primary AI Interface:** Aria
**Architecture:** Provider-Agnostic AI Orchestration
**Integration:** OneDraft Runtime / Domain Services / Command Boundary
**AI Model:** Context-Aware, Constraint-Bound, Evaluated Intelligence

---

# 1. AI ARCHITECTURE PRINCIPLE

OneDraft shall treat artificial intelligence as an orchestrated computational subsystem rather than as the underlying application itself.

Aria is the primary intelligence interface through which natural-language project requirements, design instructions, model queries, documentation requests, compliance questions, redline instructions and revision requests are interpreted and transformed into structured OneDraft operations.

The AI subsystem shall never become the authoritative system of record.

The authoritative project state remains within the OneDraft project model, revision system, compliance manifests, validation results, documentation system and acceptance records.

The AI architecture therefore operates as:

```
USER
 ↓
ARIA
 ↓
AI ORCHESTRATOR
 ↓
CONTEXT CONSTRUCTION
 ↓
MODEL PROVIDER ADAPTER
 ↓
AI MODEL
 ↓
STRUCTURED RESPONSE
 ↓
COMMAND INTERPRETATION
 ↓
DOMAIN COMMAND BOUNDARY
```

```
  ↓
VALIDATION
  ↓
PROJECT MODEL
```

---

# 2. AI SUBSYSTEM OBJECTIVES

The AI subsystem shall provide the intelligence required to interpret, reason over and operate upon the OneDraft computational project model.

Primary capabilities include:

```
Natural-language interpretation
Project-context reasoning
Design-intent interpretation
Project specification analysis
Model interrogation
Design proposal generation
Design modification planning
Constraint-aware reasoning
Compliance reasoning assistance
Redline interpretation
Documentation assistance
Revision analysis
Drawing assistance
Schedule assistance
Project-state explanation
Validation-result interpretation
```

The subsystem shall convert unstructured user intent into structured, verifiable application operations.

---

# 3. AI SYSTEM BOUNDARY

Aria may reason about the project and propose operations.

Aria shall not bypass the OneDraft application boundary.

The following actions shall not be directly performed by an AI model:

```
Direct SQL execution
Direct database writes
Direct filesystem replacement
Direct production configuration modification
Direct permission changes
Direct billing modification
Direct security-policy modification
Direct artifact deletion
Direct project-state mutation
```

All consequential operations must pass through controlled application capabilities.

The resulting architecture is:

```
AI
 ↓
PROPOSE
 ↓
APPLICATION CAPABILITY
 ↓
VALIDATE
 ↓
COMMIT
```

---

# 4. ARIA ORCHESTRATION LAYER

The Aria Orchestrator is the central coordination layer between OneDraft and external or internal AI models.

It shall coordinate:

```
Request intake
Authentication context
Authorization context
Project context
Conversation context
Model context
Tool availability
Prompt construction
Model selection
Inference execution
Response parsing
Structured output validation
Command generation
Confidence assessment
Constraint preservation
Execution routing
Result interpretation
Audit recording
```

The orchestrator therefore becomes the controlled AI gateway for OneDraft.

---

# 5. AI REQUEST LIFECYCLE

Every AI interaction shall follow a controlled lifecycle.

```
REQUEST
 ↓
AUTHENTICATE
 ↓
AUTHORIZE
 ↓
IDENTIFY PROJECT
```

```
 ↓
IDENTIFY REVISION
 ↓
CONSTRUCT CONTEXT
 ↓
CLASSIFY TASK
 ↓
SELECT MODEL
 ↓
BUILD PROMPT
 ↓
EXECUTE MODEL
 ↓
PARSE RESPONSE
 ↓
VALIDATE STRUCTURE
 ↓
EVALUATE RESULT
 ↓
PROPOSE COMMAND
 ↓
DOMAIN VALIDATION
 ↓
EXECUTE OR RETURN PROPOSAL
 ↓
RECORD RESULT
```

The lifecycle must preserve project and revision provenance.

---

# 6. AI REQUEST MODEL

Every AI request shall receive a unique identifier.

Conceptually:

```
ai_request
```

Fields:

```
id UUID
project_id UUID
user_id UUID
session_id UUID
revision_id UUID
model_version_id UUID
request_type VARCHAR
message TEXT
context_reference JSONB
provider VARCHAR
model VARCHAR
status VARCHAR
created_at TIMESTAMPTZ
completed_at TIMESTAMPTZ
```

The request record establishes traceability between the user instruction, AI execution and resulting application action.

---

# 7. AI REQUEST TYPES

Initial request types shall include:

```
PROJECT_INTAKE
PROJECT_QUERY
DESIGN_REQUEST
DESIGN_MODIFICATION
MODEL_QUERY
MODEL_MODIFICATION
REDLINE_INTERPRETATION
DOCUMENTATION_REQUEST
COMPLIANCE_QUERY
VALIDATION_QUERY
REVISION_ANALYSIS
DRAWING_QUERY
SCHEDULE_QUERY
PROJECT_REVIEW
GENERAL_PROJECT_ASSISTANCE
```

Additional request types may be introduced without changing the fundamental orchestration architecture.

---

# 8. AI SESSION MODEL

An AI session provides conversational continuity for a user working within OneDraft.

A session may reference:

```
organization
user
project
revision
model version
drawing
view
redline
active task
```

The session is contextual state.

It is not authoritative project state.

The authoritative state remains within the project model and associated persistence systems.

---

# 9. CONVERSATIONAL CONTEXT

Aria shall construct context from authoritative OneDraft sources rather than relying exclusively upon conversational history.

Relevant context may include:

```
Project requirements
Site information
Building information
Levels
Spaces
Elements
Assemblies
Constraints
Relationships
Current revision
Previous revisions
Active redlines
Code manifest
Validation results
Drawing metadata
Schedules
Project status
User instructions
Current task
```

Only context relevant to the requested operation should be supplied to the model.

---

# 10. CONTEXT CONSTRUCTION ENGINE

The Context Construction Engine shall assemble the information required for an AI request.

Conceptually:

```
PROJECT
 +
CURRENT REVISION
 +
MODEL STATE
 +
ACTIVE CONSTRAINTS
 +
RELEVANT DOCUMENTATION
 +
COMPLIANCE CONTEXT
 +
USER REQUEST
 ↓
CONTEXT PACKAGE
```

The context package shall be deterministic where practical.

The system should record the context references used for consequential AI operations.

---

# 11. CONTEXT PRIORITY

When conflicting information exists, context shall be resolved according to authority.

Initial precedence:

```
Current Project Model
 ↓
Active Revision
 ↓
Explicit Project Requirements
 ↓
Active Constraints
 ↓
Validated Compliance Information
 ↓
Validated Documentation
 ↓
User Conversation
 ↓
AI Inference
```

AI inference shall not silently override authoritative project information.

---

# 12. PROMPT CONSTRUCTION

Prompts shall be generated through controlled templates and context assembly rather than uncontrolled string concatenation.

Prompt construction may contain:

```
System instructions
OneDraft operating rules
Project context
Task definition
Relevant model state
Constraints
Compliance context
Required output schema
Execution restrictions
Validation requirements
User instruction
```

The final prompt should be reproducible for audit and evaluation purposes where permitted by provider policies and applicable privacy requirements.

---

# 13. PROMPT VERSIONING

Prompt templates shall be versioned.

Conceptually:

```
prompt_template_id
prompt_version
template_hash
created_at
status
```

A consequential AI request should record the prompt version used.

This permits comparison between AI behavior across software releases.

---

# 14. MODEL PROVIDER ABSTRACTION

OneDraft shall not bind the application directly to a single AI provider.

The architecture shall use a provider adapter interface.

Conceptually:

```
Aria Orchestrator
        ↓
AI Provider Interface
        ↓
Provider Adapter
        ↓
Model Provider
```

Provider adapters may support:

```
Hosted commercial models
Enterprise models
Private models
Local models
Future model providers
Specialized inference services
```

---

# 15. PROVIDER ADAPTER CONTRACT

Each provider adapter shall expose a common internal interface.

Conceptually:

```
generate()
stream()
embed()
```

```
classify()
validate()
estimate_tokens()
get_capabilities()
```

Provider-specific APIs remain behind the adapter.

The rest of OneDraft interacts with the normalized interface.

---

# 16. MODEL REGISTRY

OneDraft shall maintain an internal model registry.

Conceptually:

```
ai.model_registry
```

Fields may include:

```
id UUID
provider VARCHAR
model_name VARCHAR
model_version VARCHAR
capabilities JSONB
context_limit INTEGER
supports_structured_output BOOLEAN
supports_tool_calling BOOLEAN
supports_streaming BOOLEAN
status VARCHAR
created_at TIMESTAMPTZ
updated_at TIMESTAMPTZ
```

The registry allows the orchestrator to select models according to task requirements.

---

# 17. MODEL CAPABILITY MATRIX

Models shall be evaluated according to capability rather than simply provider identity.

Example capabilities:

```
reasoning
structured_output
tool_calling
long_context
vision
document_analysis
code_generation
classification
embedding
low_latency
high_accuracy
```

The orchestrator may use this matrix to determine the appropriate model for a task.

---

# 18. MODEL SELECTION

Model selection shall consider:

```
Task type
Required capability
Context size
Latency requirements
Accuracy requirements
Structured-output requirements
Cost constraints
Availability
Project sensitivity
Organization policy
Provider availability
```

The selected model becomes part of the AI request provenance.

---

# 19. MODEL ROUTING

The routing layer may select different models for different operations.

For example:

```
Simple project query
 ↓
Low-latency model

Complex design reasoning
 ↓
High-reasoning model

Document extraction
 ↓
Document-capable model

Geometry command interpretation
 ↓
Structured-output model

Embedding/search
 ↓
Embedding model
```

The routing architecture remains provider-independent.

---

# 20. STRUCTURED AI OUTPUT

Where Aria is expected to produce an application operation, the output shall conform to a defined schema.

Conceptually:

```
{
  "operation": "MOVE_ELEMENT",
  "targets": ["uuid"],
  "parameters": {
    "distance": 300,
    "unit": "mm",
    "direction": "NORTH"
  },
  "constraints": ["uuid"],
  "reason": "Maintain exterior envelope.",
  "confidence": 0.94
}
```

The application validates this structure before any command reaches the domain layer.

---

# 21. AI COMMAND TRANSLATION

Natural-language instructions shall be translated into OneDraft command primitives.

Example:

```
"Move the rear bedroom wall 300 mm north."
```

becomes:

```
MOVE_ELEMENT
```

with:

```
target
distance
unit
direction
preserved_constraints
```

The resulting command is then processed by the established Project Model command boundary.

---

# 22. AI DOES NOT DEFINE GEOMETRY STATE

Aria may request a geometric operation.

The Geometry & Parametric Model Engine determines whether that operation is mathematically and structurally valid.

The architecture is therefore:

```
ARIA
 ↓
GEOMETRIC COMMAND
 ↓
PROJECT MODEL
 ↓
GEOMETRY ENGINE
 ↓
GEOMETRY VALIDATION
 ↓
COMMIT
```

Aria does not become the geometry engine.

---

# 23. AI DOES NOT DEFINE COMPLIANCE STATE

Aria may interpret regulatory information or assist with compliance reasoning.

The Compliance & Regulatory Engine remains authoritative for regulatory manifests and rule evaluation.

The architecture is:

```
ARIA
 ↓
COMPLIANCE QUERY / PROPOSAL
 ↓
COMPLIANCE ENGINE
 ↓
RULE EVALUATION
 ↓
VALIDATION RESULT
```

AI reasoning cannot silently establish a regulatory requirement as fact.

---

# 24. AI DOES NOT DEFINE VALIDATION RESULTS

Aria may interpret validation findings.

The Validation & Verification Engine remains authoritative for validation execution.

Therefore:

```
VALIDATION ENGINE
 ↓
VALIDATION RESULT
 ↓
ARIA INTERPRETATION
```

rather than:

```
ARIA
 ↓
VALIDATION RESULT
```

This distinction preserves deterministic verification.

---

# 25. AI CONFIDENCE

AI-generated operations may contain a confidence value.

Confidence is informational.

It shall not replace deterministic validation.

Conceptually:

```
AI confidence
+
schema validation
+
constraint validation
+
geometry validation
+
compliance validation
+
coordination validation
```

determine whether an operation may proceed.

High AI confidence does not automatically authorize execution.

---

# 26. AI ACTION MODES

Initial Aria operating modes shall include:

```
ASSIST
PROPOSE
EXECUTE
REVIEW
ANALYZE
EXPLAIN
```

The mode determines how the resulting AI operation is handled.

For example:

```
ASSIST
→ informational response

PROPOSE
→ structured proposed command

EXECUTE
→ command submitted to application boundary

REVIEW
→ analysis of existing project state

EXPLAIN
→ interpretation of existing authoritative information
```

---

# 27. EXECUTION AUTHORIZATION

An AI-generated command requiring project mutation must satisfy:

```
authenticated user
+
authorized project membership
+
authorized AI capability
+
valid current revision
+
valid command schema
+
constraint verification
+
domain validation
```

Only then may the command be committed.

---

# 28. HUMAN APPROVAL BOUNDARY

Certain operations may require explicit human approval.

Examples include:

```
major design changes
final document generation
final acceptance
professional review
regulated submissions
irreversible operations
organization-level configuration
```

The AI subsystem must respect these approval boundaries.

---

# 29. TOOL-CALL ARCHITECTURE

Aria may interact with OneDraft through controlled tools.

Conceptually:

```
ARIA
 ↓
TOOL REGISTRY
 ↓
TOOL
 ↓
DOMAIN SERVICE
 ↓
RESULT
 ↓
ARIA
```

Tools may include:

```
get_project
get_model
get_element
search_elements
get_constraints
get_validation_results
get_drawing
create_design_proposal
create_command
run_validation
generate_drawing
create_redline
get_revision
```

Each tool must have an explicit schema and authorization boundary.

---

# 30. TOOL REGISTRY

Tools shall be registered with:

```
tool_id
name
description
input_schema
output_schema
required_scope
risk_level
execution_mode
version
status
```

The orchestrator shall only expose tools permitted for the current request.

---

# 31. AI TOOL PERMISSIONING

Tool availability shall be determined by:

```
user permissions
organization policy
project permissions
request mode
tool risk
current project state
```

A model must not infer permission merely because a tool exists.

---

# 32. AI MEMORY ARCHITECTURE

AI memory shall be separated into distinct categories.

```
Conversation Context
Project Context
Task Context
Persistent Project Data
System Configuration
```

Persistent project information belongs in the OneDraft data model.

Conversation history does not become authoritative merely because Aria has previously seen it.

---

# 33. RETRIEVAL ARCHITECTURE

Large project datasets shall not necessarily be placed entirely into every model context.

The system may retrieve relevant information using:

```
project queries
object queries
semantic retrieval
document retrieval
revision retrieval
constraint retrieval
code-rule retrieval
validation retrieval
```

The retrieved information is assembled into the active context package.

---

# 34. EMBEDDING AND SEMANTIC RETRIEVAL

Where required, OneDraft may maintain embeddings for:

```
project requirements
documents
notes
code references
design descriptions
specifications
redlines
historical decisions
```

Embeddings are derived search infrastructure.

They are not authoritative project state.

---

# 35. AI DOCUMENT UNDERSTANDING

Aria may process project documents through controlled document-analysis services.

Supported sources may include:

```
PDF
text
specifications
survey information
existing drawings
project notes
regulatory documents
manufacturer information
```

Extracted information shall be identified as source-derived information until verified.

---

# 36. AI VISION CAPABILITY

Where supported, the AI subsystem may process visual information such as:

```
architectural drawings
redlines
site photographs
existing-condition photographs
scanned documents
diagrammatic information
```

Visual interpretation must remain distinguishable from authoritative geometric model state.

---

# 37. AI OUTPUT CLASSIFICATION

AI outputs shall be classified before application use.

Initial classifications:

```
INFORMATIONAL
ANALYTICAL
PROPOSAL
COMMAND
VALIDATION_INTERPRETATION
DOCUMENTATION_ASSISTANCE
USER_ACTION_REQUIRED
ERROR
```

The classification determines the appropriate downstream workflow.

---

# 38. AI RESPONSE VALIDATION

AI responses intended for machine processing shall pass structural validation.

Validation shall include:

```
schema validation
required-field validation
type validation
enum validation
identifier validation
project-scope validation
revision validation
permission validation
```

Invalid outputs shall not reach the domain command boundary.

---

# 39. AI DETERMINISM

AI inference itself may not be mathematically deterministic.

OneDraft therefore establishes determinism around consequential operations.

The system must deterministically validate:

```
command schema
target existence
revision state
constraints
geometry
compliance
coordination
authorization
transaction state
```

The goal is:

```
NON-DETERMINISTIC REASONING
+
DETERMINISTIC APPLICATION VALIDATION
=
CONTROLLED AI EXECUTION
```

---

# 40. AI EVALUATION INTERFACE

AI models shall be evaluated independently from production execution.

Evaluation criteria may include:

```
instruction following
structured-output accuracy
command accuracy
constraint preservation
project-context accuracy
hallucination rate
tool-selection accuracy
regulatory reasoning accuracy
geometry-command accuracy
response latency
cost
```

The evaluation subsystem shall be further specified in the dedicated AI Evaluation and Guardrails stack.

---

# 41. MODEL VERSION PROVENANCE

Every consequential AI request should identify:

```
provider
model
model version
prompt version
context version
tool versions
orchestrator version
application version
```

This establishes reproducibility.

---

# 42. AI REQUEST HASHING

Where appropriate, the system may compute hashes over:

```
request payload
context package
prompt package
structured output
command payload
```

These hashes provide integrity references without requiring the database to duplicate every source artifact.

---

# 43. AI AUDIT EVENTS

AI activity shall produce auditable events.

Examples:

```
AI_REQUEST_CREATED
AI_CONTEXT_BUILT
AI_MODEL_SELECTED
AI_REQUEST_STARTED
AI_RESPONSE_RECEIVED
AI_RESPONSE_VALIDATED
AI_COMMAND_PROPOSED
AI_COMMAND_APPROVED
AI_COMMAND_REJECTED
AI_COMMAND_EXECUTED
AI_REQUEST_FAILED
```

These events integrate with the established OneDraft event bus and audit architecture.

---

# 44. AI ERROR HANDLING

AI failures shall be classified.

Initial categories:

```
PROVIDER_UNAVAILABLE
MODEL_UNAVAILABLE
CONTEXT_FAILURE
SCHEMA_FAILURE
TIMEOUT
RATE_LIMITED
AUTHORIZATION_FAILURE
TOOL_FAILURE
VALIDATION_FAILURE
COMMAND_FAILURE
UNKNOWN_ERROR
```

The orchestrator shall return a controlled error rather than exposing provider-specific implementation details.

---

# 45. PROVIDER FAILOVER

Where multiple compatible providers exist, the orchestrator may support controlled failover.

Conceptually:

```
PRIMARY MODEL
 ↓
FAILURE
 ↓
COMPATIBLE SECONDARY MODEL
 ↓
RETRY
```

Failover shall preserve:

```
request identity
project identity
revision identity
authorization
context
command schema
audit trail
```

A provider change shall not silently change the project revision.

---

# 46. RATE LIMITING

AI requests shall be subject to platform and organization-level controls.

Limits may be applied to:

```
requests per minute
requests per hour
tokens
concurrent requests
model-specific usage
organization usage
project usage
```

These controls integrate with the Billing, Entitlement and API Protection subsystems.

---

# 47. COST ACCOUNTING

AI usage shall be measurable.

The system may record:

```
provider
model
input tokens
output tokens
estimated cost
request duration
organization
project
user
request type
```

Usage records may feed the billing and entitlement subsystem.

---

# 48. DATA PRIVACY BOUNDARY

AI requests shall only expose information permitted by:

```
user authorization
organization policy
project access
data classification
provider policy
```

Sensitive project information shall not be transmitted to an external provider unless permitted by the applicable deployment and data-governance configuration.

---

# 49. PROVIDER POLICY

Each AI provider configuration shall define:

```
data handling policy
retention policy
supported capabilities
regional availability
security requirements
maximum context size
rate limits
cost model
availability
```

The provider adapter shall enforce provider-specific requirements.

---

# 50. LOCAL MODEL SUPPORT

The architecture shall permit locally hosted models.

Conceptually:

```
Aria
 ↓
Provider Interface
 ↓
Local Provider Adapter
 ↓
Local Inference Service
```

This permits future deployment models where project information must remain within controlled infrastructure.

---

# 51. AI MODEL SELECTION POLICY

The orchestrator shall prefer the least complex model capable of reliably performing the requested task, subject to:

```
accuracy
context requirements
security
latency
availability
cost
structured-output capability
```

Complex reasoning shall not automatically require the most expensive available model.

---

# 52. CONTEXT WINDOW MANAGEMENT

The Context Construction Engine shall manage model context limits.

Where context exceeds the model's available capacity, the system may use:

```
retrieval
summarization
hierarchical context
object filtering
revision filtering
document chunking
semantic search
```

Authoritative source references shall remain available for retrieval.

---

# 53. CONTEXT IMMUTABILITY FOR COMMANDS

For consequential commands, the context used to generate the command shall be recorded sufficiently to establish what project state Aria operated upon.

A command generated against:

```
Revision 17
```

must not be silently applied against:

```
Revision 18
```

without revalidation.

---

# 54. STALE COMMAND PROTECTION

Before execution, the system verifies:

```
expected_revision == current_revision
```

If not:

```
COMMAND_REQUIRES_REVALIDATION
```

The AI request may then be reconstructed against the current state.

---

# 55. AI COMMAND LIFECYCLE

The standard command lifecycle is:

```
AI REQUEST
 ↓
INTERPRETATION
 ↓
PROPOSAL
 ↓
STRUCTURAL VALIDATION
 ↓
AUTHORIZATION
 ↓
CONSTRAINT CHECK
 ↓
DOMAIN EXECUTION
 ↓
GEOMETRY VALIDATION
 ↓
COMPLIANCE VALIDATION
 ↓
REVISION CREATION
 ↓
DOCUMENTATION IMPACT
 ↓
AUDIT EVENT
```

This maintains consistency with the established OneDraft architecture.

---

# 56. AI DESIGN PROPOSALS

When a user requests design assistance rather than direct execution, Aria may create a design proposal.

A proposal may contain:

```
description
affected_objects
proposed_changes
rationale
constraints
estimated_impacts
confidence
```

The proposal remains non-authoritative until accepted through the appropriate application workflow.

---

# 57. MULTI-STEP AI OPERATIONS

Complex tasks may require multiple AI and application operations.

Example:

```
USER REQUEST
 ↓
ARIA
 ↓
READ PROJECT
 ↓
ANALYZE CONSTRAINTS
 ↓
PROPOSE DESIGN
 ↓
RUN VALIDATION
 ↓
REVISE PROPOSAL
 ↓
CREATE COMMAND
 ↓
EXECUTE
 ↓
REGENERATE DRAWINGS
 ↓
REPORT RESULT
```

Each consequential operation must retain its own provenance.

---

# 58. AI TASK PLANNING

Aria may decompose complex user requests into smaller application tasks.

The task plan shall be represented structurally.

Conceptually:

```
plan_id
project_id
request_id
steps[]
dependencies[]
status
```

Each step must reference a defined application capability.

Aria may plan.

The application executes.

---

# 59. AI TOOL EXECUTION BOUNDARY

The model may request a tool.

The orchestrator determines whether the tool may execute.

```
MODEL
 ↓
TOOL REQUEST
 ↓
ORCHESTRATOR
 ↓
AUTHORIZATION
 ↓
SCHEMA VALIDATION
 ↓
DOMAIN SERVICE
 ↓
RESULT
 ↓
MODEL
```

The model does not directly invoke infrastructure.

---

# 60. AI RESPONSE STREAMING

Streaming may be supported for user-facing conversational responses.

Streaming shall not imply incremental project mutation.

Project mutations remain transactional.

Therefore:

```
STREAMING RESPONSE
```

and:

```
TRANSACTIONAL COMMAND
```

remain separate mechanisms.

---

# 61. AI CACHING

Non-authoritative AI responses may be cached where appropriate.

Caching shall not be used to bypass project-state validation.

A cached response referencing project state must identify the context or revision from which it was generated.

---

# 62. AI OBSERVABILITY

AI execution shall integrate with the OneDraft Observability subsystem.

Metrics may include:

```
request count
latency
token usage
provider latency
model latency
failure rate
tool-call rate
command success rate
validation rejection rate
```

Tracing shall connect:

```
API request
 ↓
AI request
 ↓
model invocation
 ↓
tool calls
 ↓
domain command
 ↓
validation
 ↓
revision
```

---

# 63. AI SECURITY

The AI subsystem shall protect against:

```
prompt injection
tool abuse
context poisoning
unauthorized data access
cross-project context leakage
malicious document instructions
unsafe command generation
privilege escalation
provider credential exposure
```

AI-generated instructions shall never override application authorization.

---

# 64. PROMPT INJECTION DEFENSE

External project documents and user-provided content shall be treated as data unless explicitly designated as trusted system instructions.

The architecture shall distinguish:

```
SYSTEM INSTRUCTIONS
PLATFORM RULES
APPLICATION CONTEXT
PROJECT DATA
USER INSTRUCTIONS
EXTERNAL DOCUMENT CONTENT
AI OUTPUT
```

Lower-trust content shall not override higher-priority operating rules.

---

# 65. CROSS-PROJECT ISOLATION

AI context construction must enforce project isolation.

A request for:

```
Project A
```

must never retrieve unauthorized information from:

```
Project B
```

even where semantic retrieval produces a potentially similar result.

Tenant and project authorization must be enforced before context assembly.

---

# 66. AI DATA PROVENANCE

Information supplied to Aria should retain source references where practical.

Example:

```
statement
source_type
source_id
revision_id
document_id
confidence
```

This permits Aria responses to distinguish:

```
authoritative project fact
```

```
validated result
user-provided requirement
external source
AI inference
```

---

# 67. AI REASONING BOUNDARY

The internal reasoning process of the model is not itself the project record.

The application records:

```
request
context references
structured response
command
validation
result
```

rather than depending upon hidden model reasoning as an authoritative state mechanism.

---

# 68. AI EXPLANATION

Aria shall be capable of explaining application results using authoritative project information.

For example:

```
Why was this wall moved?
```

The response may reference:

```
command
revision
constraint
validation result
user instruction
```

The explanation should distinguish recorded facts from AI interpretation.

---

# 69. AI RECOMMENDATIONS

Recommendations shall be represented as recommendations rather than silently converted into project state.

Example:

```
RECOMMENDATION
```

```
 ↓
USER REVIEW
 ↓
ACCEPT
 ↓
COMMAND
 ↓
VALIDATE
 ↓
COMMIT
```

This preserves the human review boundary where required.

---

# 70. AI MODEL TESTING

Every supported model and provider adapter shall be tested against representative OneDraft workloads.

Testing categories include:

```
simple queries
complex design requests
structured commands
constraint preservation
revision awareness
document interpretation
redline interpretation
compliance assistance
tool selection
error handling
```

The detailed evaluation framework is defined by the subsequent AI Evaluation and Guardrails subsystem.

---

# 71. AI PROVIDER CONTRACT TESTING

Provider adapters shall satisfy a common contract.

Tests shall verify:

```
authentication
request construction
response parsing
structured output
streaming
timeouts
rate limits
errors
capability reporting
```

A provider adapter that fails the common contract cannot become production eligible.

---

# 72. MODEL REGISTRATION

New models shall enter the platform through controlled registration.

The registration process shall establish:

```
provider
model
version
capabilities
context limits
structured-output support
tool support
security classification
status
evaluation status
```

Models shall not automatically become eligible for consequential project operations.

---

# 73. MODEL PROMOTION

Models may progress through:

```
REGISTERED
 ↓
TESTING
 ↓
EVALUATED
 ↓
APPROVED
 ↓
PRODUCTION
 ↓
DEPRECATED
```

This creates controlled model lifecycle management.

---

# 74. MODEL DEPRECATION

When a model is deprecated:

```
existing historical records remain intact
```

Historical AI requests retain the provider and model information originally used.

New requests may be routed to an approved successor.

---

# 75. AI CONFIGURATION

AI configuration shall include:

```
default provider
default model
fallback providers
model routing policy
context limits
token limits
temperature policy
structured-output policy
tool permissions
timeout policy
retry policy
```

Production configuration shall be managed through the established Configuration and Secrets subsystem.

---

# 76. AI CREDENTIAL MANAGEMENT

Provider credentials shall never be stored in source code.

Credentials shall be managed through:

```
secret management system
environment configuration
deployment secret store
```

The AI application receives only the credentials required for its authorized provider operations.

---

# 77. AI RETRY POLICY

Transient failures may be retried.

Retryable conditions may include:

```
temporary provider failure
network timeout
temporary rate limiting
transient infrastructure error
```

Non-retryable conditions may include:

```
authorization failure
invalid request
```

```
invalid schema
policy rejection
invalid project state
```

Retries must preserve request idempotency.

---

# 78. AI TIMEOUTS

Each AI operation shall have an explicit timeout policy.

Long-running tasks shall be delegated to the OneDraft Job & Worker Execution System rather than holding an API request indefinitely.

The architecture therefore remains:

```
API
 ↓
AI JOB
 ↓
WORKER
 ↓
AI PROVIDER
 ↓
RESULT
```

---

# 79. AI JOB INTEGRATION

The AI subsystem shall integrate with the existing job system for operations such as:

```
large document analysis
large project analysis
batch model evaluation
complex design proposal generation
multi-document reasoning
large-scale drawing analysis
```

The AI subsystem does not create a competing job architecture.

---

# 80. EVENT BUS INTEGRATION

AI lifecycle events shall publish through the established Integration & Event Bus.

Examples:

```
ai.request.created
ai.request.completed
```

```
ai.request.failed
ai.command.proposed
ai.command.approved
ai.command.rejected
ai.command.executed
ai.model.changed
ai.provider.failed
```

Downstream systems may subscribe without directly coupling to the AI orchestrator.

---

# 81. AUDIT INTEGRATION

Consequential AI activity shall be represented in the audit system.

The audit record should establish:

```
who requested the action
what project was affected
what revision was active
what model was used
what command was proposed
what validation occurred
what result was produced
when the operation occurred
```

---

# 82. AI AND DOCUMENTATION

Aria may assist the Documentation & Drawing Generation Engine by:

```
interpreting drawing requests
identifying required drawing types
suggesting views
interpreting redlines
explaining drawing results
identifying documentation impacts
```

The Documentation Engine remains responsible for actual drawing generation.

---

# 83. AI AND REDLINES

A redline may enter the AI subsystem as:

```
drawing markup
+
user instruction
```

Aria may interpret the intent and produce a structured proposal.

The resulting model modification must pass through the established command and validation systems.

---

# 84. AI AND PROJECT REVIEW

Aria may provide project review assistance by assembling:

```
current revision
validation findings
open redlines
unresolved requirements
documentation status
compliance status
acceptance status
```

The review result remains an interpretation of authoritative project records.

---

# 85. AI AND ACCEPTANCE

Aria shall not independently create customer acceptance.

Acceptance remains an explicit project workflow.

Aria may:

```
explain outstanding issues
summarize revisions
prepare review information
identify unresolved items
```

The customer or authorized reviewer performs the acceptance action.

---

# 86. AI PROVIDER INDEPENDENCE

OneDraft shall maintain a strict separation between:

```
OneDraft application logic
Aria orchestration
AI provider
AI model
```

Changing the underlying provider must not require redesigning:

```
Project Model
Geometry Engine
Compliance Engine
```

```
Validation Engine
Documentation Engine
Revision System
API Contract
```

---

# 87. AI VERSION COMPATIBILITY

The AI orchestration layer shall maintain compatibility with:

```
domain command schemas
tool schemas
project model versions
API versions
validation contracts
documentation contracts
```

When an AI command schema changes, the orchestrator must explicitly identify the compatible application version.

---

# 88. AI SAFETY STATE

Every consequential AI operation shall have a state.

Initial states:

```
RECEIVED
CONTEXTUALIZED
PLANNED
PROPOSED
VALIDATING
AUTHORIZED
EXECUTING
COMPLETED
REJECTED
FAILED
CANCELLED
```

State transitions shall be controlled by the application.

---

# 89. AI REPRODUCIBILITY

A historical AI operation shall be reconstructable to the extent permitted by provider availability and retention policies.

The system should retain:

```
request
project reference
revision reference
model reference
prompt version
context references
tool calls
structured output
command
validation results
final result
```

This supports debugging, auditing and system evaluation.

---

# 90. AI ARCHITECTURAL CONTRACT

The AI subsystem establishes the following contract:

```
ARIA
 ↓
UNDERSTANDS USER INTENT
 ↓
CONSTRUCTS PROJECT CONTEXT
 ↓
SELECTS APPROPRIATE MODEL
 ↓
GENERATES STRUCTURED OUTPUT
 ↓
PROPOSES APPLICATION OPERATIONS
 ↓
APPLICATION VALIDATES
 ↓
DOMAIN SERVICES EXECUTE
 ↓
PROJECT MODEL CHANGES
 ↓
VALIDATION CONFIRMS RESULT
 ↓
REVISION IS RECORDED
 ↓
DOCUMENTATION IS REGENERATED AS REQUIRED
```

Aria is therefore the intelligence layer, not the authority layer.

---

# 91. MVP IMPLEMENTATION COMPONENTS

The initial implementation shall include:

```
AI Orchestrator
```

```
Provider Interface
Provider Adapter
Model Registry
Context Builder
Prompt Builder
Structured Output Parser
Tool Registry
AI Request Service
AI Session Service
AI Command Translator
AI Audit Integration
AI Event Integration
```

---

# 92. INITIAL REPOSITORY STRUCTURE

The implementation may be organized as:

```
apps/
    api/
    worker/

packages/
    ai/
        orchestrator/
        providers/
        context/
        prompts/
        tools/
        schemas/
        evaluation/
        routing/

    domain/
    model/
    validation/
    documentation/
```

The exact repository structure shall remain consistent with the established OneDraft application architecture.

---

# 93. INITIAL SERVICE INTERFACES

The AI package shall expose conceptual interfaces:

```
AIOrchestrator
AIProvider
ModelRegistry
ContextBuilder
PromptBuilder
ToolRegistry
AICommandTranslator
```

`AIRequestService`

Provider implementations shall satisfy the common `AIProvider` contract.

---

# 94. INITIAL API INTERFACE

The existing API architecture shall expose controlled AI operations through endpoints such as:

```
POST /api/v1/projects/{project_id}/aria/requests
GET  /api/v1/projects/{project_id}/aria/requests/{request_id}
GET  /api/v1/projects/{project_id}/aria/sessions/{session_id}
POST /api/v1/projects/{project_id}/aria/proposals
POST /api/v1/projects/{project_id}/aria/commands
```

All mutating operations remain subject to the established authorization and project command boundaries.

---

# 95. INITIAL INTEGRATION TEST

The first AI integration test shall execute:

1. Create organization
2. Create user
3. Create project
4. Create initial requirements
5. Create building
6. Create levels
7. Create spaces
8. Create walls
9. Create doors
10. Create windows
11. Establish constraints
12. Create Revision 1
13. Submit natural-language design request
14. Construct project context
15. Select AI model

16. Generate structured response

17. Validate response schema

18. Translate response into command

19. Validate command

20. Execute command

21. Create Revision 2

22. Validate resulting model

23. Identify affected drawings

24. Record AI events

25. Retrieve complete AI request history

This becomes the first AI-to-domain integration regression test.

---

# 96. AI MVP SUCCESS CRITERIA

The subsystem is successful when Aria can:

Understand a project request

Retrieve authorized project context

Interpret project requirements

Identify relevant model objects

Read active constraints

Produce structured design proposals

Produce valid domain commands

Respect revision boundaries

Use authorized tools

Route requests through an abstract provider interface

Record model provenance

Survive provider failures

Return structured errors

Create auditable AI events

```
Operate without direct database access

Operate without direct infrastructure access
```

---

# 97. ARCHITECTURAL INVARIANTS

The following invariants shall remain permanent:

```
AI is not the system of record.

AI does not directly modify PostgreSQL.

AI does not bypass domain services.

AI does not bypass authorization.

AI does not bypass validation.

AI does not silently modify project revisions.

AI cannot override active constraints.

AI cannot establish regulatory truth independently.

AI cannot create customer acceptance independently.

AI providers remain replaceable.

Project provenance remains authoritative.
```

---

# 98. RELATIONSHIP TO PRECEDING SUBSYSTEMS

This subsystem builds directly upon:

```
Project Architecture
Domain Model
Database & OpenAPI Contract
Project Model
Geometry Engine
Compliance Engine
Validation Engine
Documentation Engine
Review & Acceptance Engine
Runtime Orchestrator
Job & Worker System
Artifact & Storage System
Integration & Event Bus
Identity & Security System
Observability System
```

No preceding architecture is redefined.

The AI layer becomes the intelligence interface across those established services.

---

# 99. RELATIONSHIP TO FOLLOWING SUBSYSTEM

The next subsystem shall establish the dedicated:

```
AI Evaluation, Guardrails & Deterministic Command Verification System
```

That subsystem shall formalize:

```
AI evaluation datasets
model benchmarking
guardrails
command safety
confidence thresholds
deterministic verification
AI regression testing
prompt-injection testing
tool-use verification
model release gates
```

The present document establishes the AI intelligence and provider architecture upon which that evaluation system operates.

---

# 100. FINAL ARCHITECTURAL DECISION

OneDraft shall not embed intelligence directly into the Project Model, Geometry Engine, Compliance Engine or Documentation Engine.

Instead:

```
ONE DRAFT
        ↓
      ARIA
        ↓
AI ORCHESTRATION
        ↓
MODEL PROVIDER ABSTRACTION
        ↓
STRUCTURED INTELLIGENCE
        ↓
APPLICATION CAPABILITIES
        ↓
DOMAIN SERVICES
        ↓
DETERMINISTIC VALIDATION
```

```
        ↓
VERSIONED PROJECT STATE
```

This architecture allows OneDraft to evolve its AI capabilities independently from its authoritative computational project model.

Aria therefore functions as the **intelligence and orchestration interface of OneDraft**, while the underlying OneDraft application remains the authoritative computational system.

---

# 101. SPECIFICATION STATUS

**ONEDRAFT AI INTELLIGENCE & MODEL PROVIDER ARCHITECTURE v0.1**

**STATUS: ESTABLISHED**

The AI architecture now defines:

```
Aria orchestration
AI request lifecycle
AI sessions
Context construction
Prompt construction
Model registry
Model selection
Provider abstraction
Provider adapters
Structured output
Tool execution
AI command translation
AI confidence
AI provenance
AI security
AI privacy boundaries
AI observability
AI auditing
AI event integration
AI job integration
AI versioning
AI provider failover
AI reproducibility
AI execution boundaries
```